

Variational Inference Notes

jasonxiong789

March 2025

1 Variational Dequantization

Note the heavy abuse of notation (to match the flow ++ paper), where p_{model} just the general joint distribution across all variables learned.

$$\begin{aligned}
Op_{model}(u|x) &= \frac{p_{model}(u, x)}{P_{model}(x)} \\
&= \frac{p_{model}(x + u)}{\int_{[0,1)^D} p_{model}(x + u)} \\
D_{KL}(q(u|x)||p_{model}(u|x)) &= \int_{[0,1)^D} q(u|x) \log \left(\frac{q(u|x)}{p_{model}(u|x)} \right) du \\
&= \int_{[0,1)^D} q(u|x) \log \left(\frac{q(u|x)}{p_{model}(x + u)} P_{model}(x) \right) du \\
&= \int_{[0,1)^D} q(u|x) \log (P_{model}(x)) du - \int_{[0,1)^D} q(u|x) \log \left(\frac{p_{model}(x + u)}{q(u|x)} \right) du \\
&= \log (P_{model}(x)) \int_{[0,1)^D} q(u|x) du - \int_{[0,1)^D} q(u|x) \log \left(\frac{p_{model}(x + u)}{q(u|x)} \right) du \\
&= \log (P_{model}(x)) - \int_{[0,1)^D} q(u|x) \log \left(\frac{p_{model}(x + u)}{q(u|x)} \right) du
\end{aligned}$$

Thus

$$\begin{aligned}
\mathbb{E}_{x \sim P_{data}} [D_{KL}(q(u|x)||p_{model}(u|x))] &= \mathbb{E}_{x \sim P_{data}} \left[\log (P_{model}(x)) - \int_{[0,1)^D} q(u|x) \log \left(\frac{p_{model}(x + u)}{q(u|x)} \right) du \right] \\
&= \mathbb{E}_{x \sim P_{data}} [\log (P_{model}(x))] - \mathbb{E}_{x \sim P_{data}} \left[\int_{[0,1)^D} q(u|x) \log \left(\frac{p_{model}(x + u)}{q(u|x)} \right) du \right] \\
&= \mathbb{E}_{x \sim P_{data}} [\log (P_{model}(x))] - \mathbb{E}_{x \sim P_{data}} \left[\mathbb{E}_{u \sim q(\cdot|x)} \left[\log \left(\frac{p_{model}(x + u)}{q(u|x)} \right) \right] \right]
\end{aligned}$$

2 Evidence Lower Bound (ELBO)

$$\begin{aligned}
\log(p(x)) &= \mathbb{E}_{z \sim q(z)} [\log(p(x))] \\
&= \mathbb{E}_{z \sim q(z)} \left[\log \left(\frac{q(z)p(z|x)p(x)}{q(z)p(z|x)} \right) \right] \\
&= \mathbb{E}_{z \sim q(z)} \left[\log \left(\frac{q(z)p(x, z)}{q(z)p(z|x)} \right) \right] \\
&= \mathbb{E}_{z \sim q(z)} \left[\log \left(\frac{q(z)}{p(z|x)} \right) + \log \left(\frac{p(x, z)}{q(z)} \right) \right] \\
&= D_{KL}(q(z)||p(z|x)) + \mathbb{E}_{z \sim q(z)} \left[\log \left(\frac{p(x, z)}{q(z)} \right) \right] \\
&= D_{KL}(q(z)||p(z|x)) + \mathbb{E}_{z \sim q(z)} [\log(p(x, z))] - \mathbb{E}_{z \sim q(z)} [\log(q(z))] \\
&= D_{KL}(q(z)||p(z|x)) + \text{ELBO}
\end{aligned}$$

Note that when $q(z)$ has the same distribution as $p(z|x)$, maximizing the ELBO is the same as maximizing the original objective $\log(p(x))$ (e.g. the evidence). In addition, ELBO is a lower bound on the original objective since $D_{KL}(q(z)||p(z|x)) \geq 0$. Note that this holds for any arbitrary $q(z)$ as long as the distributions for $p(x)$ and $p(z|x)$ (which are generally parameterized) are coherent with each other (we also need $p(x|z)$ to be coherent, but we assume it always is to match however we define $p(x)$ and $p(z|x)$, which is why it is intractable and we do not try to assume its distribution or parameterize it). Note that optimizing the parameters of q (which are assumed to not share any parameters with the actual distributions involving the real distributions $p(x)$, $p(x|z)$ or $p(z|x)$) will not impact the real distributions.

Here is another more common derivation of the ELBO:

$$\begin{aligned}
\log(p(x)) &= \log \left(\int_z p(x, z) \right) \\
&= \log \left(\int_z q(z) \frac{p(x, z)}{q(z)} \right) \\
&= \log \left(\mathbb{E}_{z \sim q(z)} \left[\frac{p(x, z)}{q(z)} \right] \right) \\
&\geq \mathbb{E}_{z \sim q(z)} \left[\log \left(\frac{p(x, z)}{q(z)} \right) \right] \\
&= \mathbb{E}_{z \sim q(z)} [\log(p(x, z))] - \mathbb{E}_{z \sim q(z)} [\log(q(z))] \\
&= \text{ELBO}
\end{aligned}$$

where the inequality is from applying Jensen's inequality on $\log$, which is concave. Typically, $q(z)$ is written as $q_\phi(z|x)$ and we have another network written as $p_\theta(x|z)$. $p(z)$ can may or may not be parameterized, and if it is, we can denote it as $p_\theta(z)$. As long as $p_\theta(x|z)$ and $p_\theta(z)$ are property distributions (e.g. probability sums to 1), the ELBO always lower bounds $\log(p_\theta(x))$ for any $q(z)$. Looking more closely at the ELBO

$$\begin{aligned}
\text{ELBO} &= \mathbb{E}_{z \sim q_\phi(z|x)} \left[\log \left(\frac{p_\theta(x, z)}{q_\phi(z|x)} \right) \right] \\
&= \mathbb{E}_{z \sim q_\phi(z|x)} \left[\log \left(\frac{p_\theta(x|z)p_\theta(z)}{q_\phi(z|x)} \right) \right] \\
&= \mathbb{E}_{z \sim q_\phi(z)} [p_\theta(x|z)] + \mathbb{E}_{z \sim q_\phi(z)} \left[\log \left(\frac{p_\theta(z)}{q_\phi(z)} \right) \right] \\
&= \mathbb{E}_{z \sim q_\phi(z)} [p_\theta(x|z)] - D_{KL}(q_\phi(z)||p_\theta(z)) \\
&= \mathbb{E}_{z \sim q_\phi(z)} [p_\theta(x|z)] + \mathbb{E}_{z \sim q_\phi(z)} [p_\theta(z)] - \mathbb{E}_{z \sim q_\phi(z)} [q_\phi(z)] \\
&= \mathbb{E}_{z \sim q_\phi(z)} [p_\theta(x, z)] + \mathcal{H}(q_\phi(z))
\end{aligned}$$

Maximizing the ELBO is essentially maximizing the probability that z is mapped back to x under the mapping $q_\phi(z|x)$ while the second term is like a regularization term that makes the approximate posterior $q_\phi(z|x)$ similar to the prior $p_\theta(z)$. This way we do not degenerate to the case of a flow model. For cases such as when $p_\theta(z)$ is Gaussian

or some other RV with noise, this regularization means that if we move a bit around in the embedding space, the resulting embedding should map to the same x or else you would not recover the correct x during training.

We can visualize the maximization of $[p_\theta(x, z)]$ under ϕ using a 2-D region of support with x as the x-axis, z as the y-axis, and the joint probability as the z-axis. We can think about a fixed x as vertical line and finding the z along that vertical line that maximizes the joint probability. We can also think about maximizing that term as two separate maximizations, one trying to maximize $\mathbb{E}_{z \sim q_\phi(z)} [p_\theta(x|z)]$ by trying to put all the probability of z for the largest possible value of $p_\theta(x|z)$ (choose a horizontal line that makes x the largest probability as we scan the joint probability across the horizontal line) and the other trying to maximize $p_\theta(z)$ (choose a horizontal line that maximizes the sum of joint probability across horizontal line).

Note that maximizing the ELBO with respect to θ maximizes the lower bound of $\log(p_\theta(x))$. When θ (and also $\log(p_\theta(x))$) is constant, maximizing the ELBO with respect to ϕ decreases $D_{KL}(q_\phi(z)||p|\theta(z|x))$ since

$$= D_{KL}(q_\phi(z)||p|\theta(z|x)) = \log(p_\theta(x)) - \text{ELBO}$$

Derivation for maximizing the ELBO with respect to ϕ and θ :

$$\begin{aligned} \nabla_\theta \text{ELBO} &= \nabla_\theta \mathbb{E}_{z \sim q_\phi(z|x)} [\log(p_\theta(x|z)) + \log(p_\theta(z)) - \log(q_\phi(z|x))] \\ &= \nabla_\theta \mathbb{E}_{z \sim q_\phi(z|x)} [\log(p_\theta(x|z)) + \log(p_\theta(z))] \end{aligned}$$

Here, we can use the following Monte Carlo estimate during training

$$\nabla_\theta \frac{1}{K} \sum_{k=1}^K \log(p_\theta(x|z_k)) + \log(p_\theta(z_k))$$

where z_k are sampled from $q_\phi(z|x)$.

$$\text{ELBO} = \mathbb{E}_{z \sim q_\phi(z|x)} [\log(p_\theta(x|z)) + \log(p_\theta(z)) - \log(q_\phi(z|x))]$$

$$\begin{aligned} \nabla_\phi \text{ELBO} &= \nabla_\phi \mathbb{E}_{z \sim q_\phi(z|x)} [\log(p_\theta(x|z)) + \log(p_\theta(z)) - \log(q_\phi(z|x))] \\ &= \mathbb{E}_{z \sim q_\phi(z|x)} [\nabla_\phi q_\phi(z|x) (\log(p_\theta(x|z)) + \log(p_\theta(z)) - \log(q_\phi(z|x)))] - \mathbb{E}_{z \sim q_\phi(z|x)} [\nabla_\phi \log(q_\phi(z|x))] \\ &= \mathbb{E}_{z \sim q_\phi(z|x)} [\nabla_\phi q_\phi(z|x) (\log(p_\theta(x|z)) + \log(p_\theta(z)) - \log(q_\phi(z|x)))] \end{aligned}$$

where the first step comes from the chain rule (by expanding out the expectation into integral/sum) and the second term is 0 by the EGLP lemma.

For image inputs, we typically treat the entire image as an input. For instance with CIFAR 10, x would be 32x32x3. We would then use a network to map the entire input to some $q_\phi(z|x)$ to produce some parameters μ (vector that represents mean) and σ (vector that represents log(std)) for a Gaussian with a diagonal covariance matrix. Here μ_z and σ_z would be the same dimension, a dimension that is typically smaller than the dimension of x (e.g. smaller than 32*32*3 for CIFAR 10). We would then sample a z and use a network to map the sampled z using $p_\theta(z|x)$ to produce some another set of parameters μ_x (vector that represents mean) and σ_x (vector that represents log(std)) for a Gaussian with a diagonal covariance matrix. μ_x and σ_x would both be the same dimension as the input (e.g. 32*32*3 for CIFAR 10). This effectively generate one Gaussian per rgb pixel of the original input. Sometimes, only μ_x is generated if we want only care about the highest probability sample and this μ_x is clipped between 0 and 1. This also lets us use squared error between the output and the original image since this is essentially the log likelihood of a gaussian sample where each pixel output is the mean of the gaussian and each original image pixel value is the sample for the corresponding gaussian. In this case, the std of each Gaussian is assumed to be a constant and the same for each output pixel.

Typically, we can treat the image data as continuous between 0 and 1 and just round the network output at the end when sampling. Or we can also sometimes add noise to the image data as mentioned in the dequantization above (since even though the values are real numbers between 0 and 1, they do not cover all possible values between 0 and 1).

Note the difference between autoencoder and variational-autoencoders is that autoencoders map data into a distribution instead of a fixed representation. This allows variational autoencoders to be used for sampling while autoencoders are typically just used for reconstruction.

3 EM Algorithm

We typically use an amortized approximator $q_\phi(z|x)$ for $p^\theta(z|x)$. Note we do not actually have a network parameterized by θ that gives us $p^\theta(z|x)$ when it is intractable. The θ superscript is just to show that when $p_\theta(z)$ and $p_\theta(x|z)$ have networks parameterized by θ , the $p^\theta(z|x)$ is also fixed since it must satisfy

$$p_\theta(z|x) = \frac{p(z)p_\theta(x|z)}{\int_z p(z)p^\theta(z|x)}$$

even though we do not know what $p^\theta(z|x)$ is. However, if $p^\theta(z|x)$ was tractable, we could maximize $\log(p_\theta(x))$ by fixing $q_\phi(z|x) = p^\theta(z|x)$ and just directly repeating the maximization of the ELBO

$$\begin{aligned}\theta_{k+1} &= \arg \max_{\theta'} \mathbb{E}_{z \sim p^{\theta_k}(z|x)} [\log(p_\theta(x|z)) + \log(p_\theta(z)) - \log(p^{\theta_k}(z|x))] \\ &= \arg \max_{\theta'} \mathbb{E}_{z \sim p^{\theta_k}(z|x)} [\log(p_\theta(x|z)) + \log(p_\theta(z))] \\ &= \arg \max_{\theta'} \mathbb{E}_{z \sim p^{\theta_k}(z|x)} [\log(p_\theta(x, z))]\end{aligned}$$

With each iteration, we also know that $\log(p_\theta(x))$ will increase since when $\theta' = \theta_k$, the ELBO that we are maximizing is exactly equal to $\log(p_{\theta_k}(x))$ since

$$D_{KL}(q_\phi(z|x)||p_{\theta'}(z|x)) = D_{KL}(p_{\theta_k}(z|x)||p_{\theta'}(z|x)) = 0$$

As a result, the θ_{k+1} (i.e. the argmax θ') must have an ELBO value greater than or equal to $\log(p_{\theta_k}(x))$. Since the ELBO for θ_{k+1} lower bounds $\log(p_{\theta_{k+1}}(x))$, $\log(p_{\theta_{k+1}}(x))$ must also be at least $\log(p_{\theta_k}(x))$.

4 VQ VAE

In VQ VAE, we discretize the latent variable z instead of letting it be a continuous. With continuous latents, the transitions are smoother and we can generate images that do not exist, but VQ VAE with discrete latents helps model sharper transitions and give sharper edges. In VQ-VAE, the encoder model maps each input to N a latent z represented as a vector n categorical variables $z^1, \dots, z^n$, each of which can take on K values. To do so, the encoder $q_\phi(z|x)$ produces N embedding vectors $z_q^1(x), \dots, z_q^n(x)$, each of dimension d . There is also single codebook with K vectors of $e^1, \dots, e^K$, each of dimension d , that is shared among all the $z_q^1(x), \dots, z_q^n(x)$. We then map each embedding vector $z_q^1(x), \dots, z_q^n(x)$ to its nearest neighbor in the codebook to get $z_e^1(x), \dots, z_e^n(x)$. We then pass through the $z_e^1(x), \dots, z_e^n(x)$ into the encoder to produce an output distribution with the same dimension as the input just like with the usual VAE. The loss is as follows

$$L = \log p(x|z_q(x)) + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta \|z_e(x) - \text{sg}[e]\|_2^2,$$

The first term is the ELBO loss without the KL divergence between the posterior $q_\phi(z|x)$ and $p(z)$ since the KL divergence is assumed to be a constant (e.g. KL between a deterministic $q_\phi(z|x)$ and a uniform $p(z)$). Since the gradient does not flow back through the encoder due to the code book lookup, we copy the gradients of the first term with respect to $z_e^1(x), \dots, z_e^n(x)$ as the gradient for $z_q^1(x), \dots, z_q^n(x)$. Note that we do not actually apply the gradients with respect to $z_e^1(x), \dots, z_e^n(x)$ to their respective embedding vectors. To learning the embeddings, the second term moves the embeddings closer to the encoder output. The third term moves the encoder output closer to the embeddings.

Normally, the missing KL term for the ELBO in the first term of the loss makes it so that we do not map each different x in very far apart spaces. For example, if $p(z)$ is Gaussian, all of x 's, are encouraged to be mapped close to the mean of $p(z)$. We can then sample from $p(z)$ and push to through the decoder to get a sample x . However, with VQ-VAE, sampling from a uniform prior and pushing those samples through the decoder will generally not produce a good sample x . Thus, we do a second round of training to learn the prior, by treating each encoded x $q(z|x)$ as a data point, using an autoregressive model like a transformer or pixel CNN on the encoded x . Note the encoded x is a vector of discrete latent variables, and we pass the codebook embeddings to the autoregressive model as input and each step, the auto regressive model outputs a softmax over $1, \dots, K$. Essentially, since there is no KL term, $p(z)$ and $q(z|x)$ are no longer close, so we try to learn $p(z)$ with an autoregressive model to match $q(z|x)$.

Some example dimensions for CIFAR10 would be an input of size 32*32*3 that is mapped to 8*8*10 discrete latents ($N = 640$).

5 Flow vs Latent Variable Model

We can think of a flow model where the latent variable z is continuous and

$$p(x|z) = \delta(x - f^{-1}(z)) = \delta(f^{-1}(z) - x)$$

In this case, $p(x)$ is no longer intractable and has the following closed form expression:

$$\begin{aligned} p(x) &= \int_z p(x, z) dz \\ &= \int_z p(z) p(x|z) dz \\ &= \int_z p(z) \delta(f^{-1}(z) - x) dz \end{aligned}$$

Using the change of variables

$$y = f^{-1}(z) \iff f(y) = z \implies \frac{dz}{dy} = f'(y) \implies dz = f'(y) dy$$

we get that

$$\begin{aligned} p(x) &= \int_{-\infty}^{\infty} p(x, z) dz \\ &= \int_{-\infty}^{\infty} p(z) \delta(f^{-1}(z) - x) dz \\ &= \int_{-\infty}^{\infty} p(f(y)) \delta(y - x) f'(y) dy \\ &= \int_{-\infty}^{\infty} p(f(x)) \delta(y - x) f'(x) dy \\ &= p(f(x)) f'(x) \int_{-\infty}^{\infty} \delta(y - x) dy \\ &= p(f(x)) f'(x) \end{aligned}$$

6 Auto-Encoding Variational Bayes (AEVB) Variations

The vanilla VAE maximizes the ELBO

$$\mathbb{E}_{z \sim q(z)} \left[\log \left(\frac{p(x, z)}{q(z)} \right) \right]$$

using the following Monte Carlo estimator

$$\frac{1}{L} \sum_{i=1}^L \log \left(\frac{p(x, z_i)}{q(z_i)} \right)$$

The Auto-Encoding Variational Bayes paper calls it the Stochastic Gradient Variational Bayes estimator and uses $L = 1$ in experiments.

The importance weighted auto encoder (IWAE), maximizes

$$\mathbb{E}_{z_1, z_2, \dots, z_k \sim q(z)} \left[\log \left(\frac{1}{k} \sum_{j=1}^k \frac{p(x, z_j)}{q(z_j)} \right) \right]$$

using the following Monte Carlo estimator

$$\frac{1}{L} \sum_{i=1}^L \log \left(\frac{1}{k} \sum_{j=1}^k \frac{p(x, z_{l,j})}{q(z_{l,j})} \right)$$

where, $z_{l,j}$ is the j th latent variable vector from the l th sample of k independent latent variable vectors. In the paper, $L = 1$ and k is varied. Note that this scenario where $L = 1$ is the exact same as a k sample Monte Carlo estimate of the original objective

$$\log \left(\mathbb{E}_{z \sim q(z)} \left[\frac{p(x, z)}{q(z)} \right] \right)$$

7 Reparameterization Trick

Typically, the likelihood ratio gradient is expressed as

$$\nabla \mathbb{E}_{z \sim q_\theta(z)} [f(z)] = \mathbb{E}_{z \sim q_\theta(z)} [\nabla \log(q_\theta(z)) f(z)]$$

In some cases, we can reparameterize z using a differential transformation of a different random variable. One such case is when $q_\theta(z)$ is Gaussian, with mean μ and variance Σ , we can express

$$z = \Sigma^{1/2} \epsilon + \mu$$

where $\epsilon \sim \mathcal{N}(0, I)$. Here the reparameterized expected likelihood can be expressed as

$$\mathbb{E}_{z \sim q(z)} [f(z)] = \nabla \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [f(\Sigma^{1/2} \epsilon + \mu)]$$

Thus,

$$\nabla \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [f(\Sigma_\theta^{1/2} \epsilon + \mu_\theta)] = \nabla \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [\nabla f(\Sigma_\theta^{1/2} \epsilon + \mu_\theta)]$$

Given a Gaussian z and a fixed target t , the original likelihood ratio gradient tries to shift the mean closer to each sample, but shifts more towards samples with a larger $f(z)$. For a single sample, we will always try to move the mean (increase the probability) of that sample.

The reparameterized gradient for each sample tries to shift the mean (plus some noise multiplied by the covariance) in a direction that increases $f(z)$, which typically is more controlled and has less variance. For a single sample, we will try to move the mean closer to the target. This means we can get away with much fewer samples due the decreased variance.

8 Conditional VAE

Typically in RL when learning a policy in an fully observable MDP, the optimal policy is deterministic. However, there can be multiple optimal actions for an MDP. Moreover, for behavioral cloning, training data may include multiple actions for a given state, some of which may be suboptimal but we are still ok with the model taking and want the model to copy. In these two cases, if the distribution of actions is represented with a categorical distribution, we can easily model a multi-modal probability distribution to capture multiple actions. For instance, this is why transformers with softmax output do not face this issue. However, there can be issues with continuous actions, especially if we try using a uni modal distribution like a Gaussian. For instance, in Figure 1, to avoid the tree, going left or right may be equally good. Let's say in RL case, we explore both left and right equally and both actions lead to equal rewards. Analogously, in the behavioral cloning case, let's say that both left and right actions are taken equally in the training set. However, the RL case typically does not happen since randomness in exploration will typically cause the action distribution to focus solely on left or solely on right eventually, so we will focus on the behavioural cloning case. If we use a categorical distribution to represent the policy with three actions, left, right, or straight, we could easily model a multimodal distribution that concentrates the probabilities on the left and right action. However, if the action is continuous and we try modeling the distribution with a Gaussian (e.g. more negative is more left and more positive is more right), the distribution that would be trained would have all of the probability concentrated in the middle, which is exactly what we do not want.

One simple way to resolve this is to model the output distribution of actions as a mixture of k Gaussians, where we manually pick k . In the example above, we could set $k = 1$ and let the model output the means and standard deviations of both Gaussians as well as their mixture weights as shown in Figure 3. However, the issue with this approach is that we have to manually tune k . One way we can resolve this for imitation learning is to use conditional VAEs, where the example model is in Figure 4 and the loss is given by

$$\begin{aligned}\text{Conditional ELBO} &= \mathbb{E}_{z \sim q_\phi(z|x,a)} [\log p_\theta(a|x,z) + \log p_\theta(z|x)] + \mathcal{H}(q_\phi(z|x,a)) \\ &= \mathbb{E}_{z \sim q_\phi(z|x,a)} [\log p_\theta(a|x,z)] - D_{KL}(q_\phi(z|x,a) || p_\theta(z|x)) \quad (1)\end{aligned}$$

We can think of the normal VAE as a type of unconditional VAE where x is some fixed constant or non-existent. Compared to the original ELBO essentially just replaced x with a and conditioned everything on x . During test time, when trying to decide an action for an input x , we sample a z from $p_\theta(z|x)$ and use both x and z inputs to the decoder $p_\theta(a|x,z)$ and sample an action from $p_\theta(a|x,z)$. $p_\theta(a|x,z)$ can be modeled with a simple Gaussian and the reason why this works is because, each z typically is mapped from one a during training. For example, when given the tree scenario as x , any sampled z will only correspond to the left or right action, and when we put that sampled z through the decoder with x , the output will be a Gaussian centered at either the left or right action depending on the z that was sampled. Essentially, we can think of z now as a latent space of particular actions or plans (e.g. right or left) and conditioned on z , the multimodal policy becomes a unimodal one.

For instance, if we think of a 1-D gaussian that represents z (z is usually higher dimension in practice but saying it is 1-D to make the example simpler), the part left of the mean might correspond to the left action where actions produced decoding a z from the left would lead to a unimodal distribution centered at the left action and vice versa. Essentially, when trying to learn a multimodal continuous distribution, we pass into the encoder whatever we want to predict, e.g. actions in this case, and similar predictions (similar in terms of the prediction loss function) will be mapped closer in the latent space and essentially be grouped into types. And sampling from the latent space gives us a type of the prediction, which should have a unimodal distribution for predictions of that type.

The same concept also applies for unconditional VAEs. For instance, if we have training data of blue and yellow

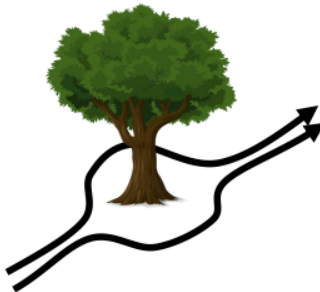


Figure 1

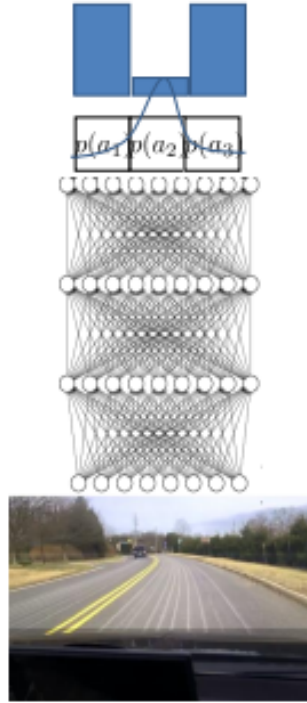


Figure 2

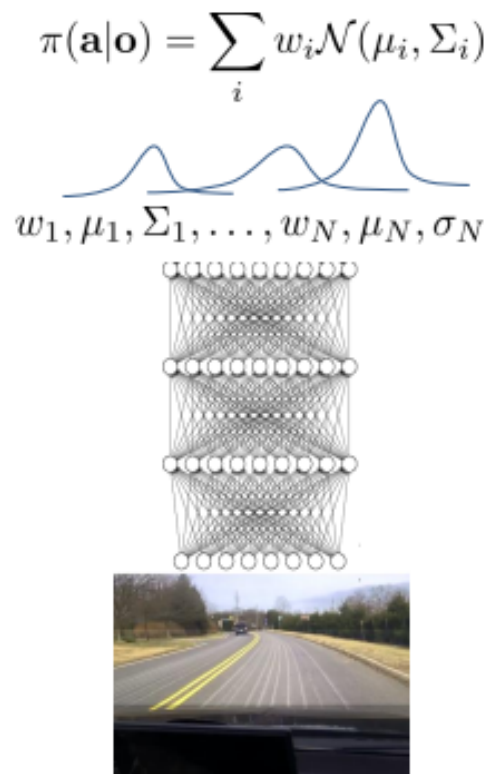


Figure 3

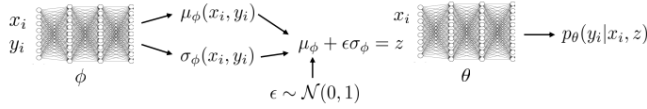


Figure 4: Enter Caption

shirts and want the model to output such shirts using a gaussian for each pixel, we sample a z at test time, which would correspond to one shirt color, and pass that z to the decoder generate an image. If we instead did not have an encoder and instead passed in the same constant input to the decoder for all examples training and used that constant input at test time (since we do not have latent space to sample from), we might instead see a green shirt (same issue as left or right action in the RL example). Essentially, passing in what we want to predict and mapping it to a latent space with fixed a simple distribution groups similar item in that simple distribution such that when we sample from the simple distribution and pass the sample to the encoder at test time, we get an output of the group the sample corresponds to.

Note that in RL imitation learning, instead of prediction one action given a state, we might try to predict a sequence of actions given a state. A sequence of actions might capture one type of behavior. For instance, if a robot is trying to put a ball into a cup, it could drop the ball in the cup or bounce it in. We could let our latent space try and capture these two behaviors such that when we sample at test time, we get one coherent behavior rather than some mixture of the two. Note that we don't necessarily need to predict a sequence to make this work since predict the one action at a time will theoretically still work since those two behaviors have separate individual actions as well, but it might be harder for the two behaviors to be captured.

Learning Latent Plans from Play Paper.

Instead of having a model that predicts a sequence actions at once, we can still have a model that predicts one action at a time, but we make it repeat the process for the same sampled latent. During training, we could pass in a sequence of actions and states $\tau = \{s_1, a_1, \dots, s_k, a_k\}$ into the decoder to get $q_\phi(z|\tau)$ and then pass in just the first state into the prior to get $p_\theta(z|s_1)$ (for the KL loss). We would then sample one z from $q_\phi(z|\tau)$. Then for each state $s_i \in \{s_1, \dots, s_k\}$ in the sequence, we would pass in s_i as well as the sampled z (we use the sample sample for all s_i in the sequence) to produce $p_\theta(a_{i+1}|s_i, z)$. We would use these $p_\theta(a_{i+1}|s_i, z)$ and construct a loss term with the real a_{i+1} for each s_i and sum them together as the reconstruction loss (first term in (1)) and add the D_{KL} between $q_\phi(z|\tau)$ and $p_\theta(z|s_1)$ to get the loss. During test time, we would sample a z from the $p_\theta(z|s)$ where s is our current state and predict one action. We would then take the action, enter a new state, and repeat, only resampling z every k action predictions (or some other time frame). Optionally, to make this a goal oriented policy, we can also condition on a goal, which is typically the last state in the sequence, which essentially changes the expressions from above to $p_\theta(a_{i+1}|s_i, g, z)$, $p_\theta(z|s_1, g)$, and $q_\phi(z|\tau)$ (this does not change since τ includes the last state).

Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware Paper.

Note that in VAEs, instead of having a neural network as an decoder and encoder, we could use a transformer encoder as the VAE encoder, where during training we pass in a sequence of states and actions (from the imitation learning dataset) to the transformer encoder and use the z produced by the CLS token. We then pass in the z along with the input x to a transformer encoder-decoder to predict a sequence of actions (we technically can also use just a transformer decoder as the VAE decoder and pass in z and x as the preset input). We then use the conditional VAE loss just as before with the D_{KL} performed against a standard normal. During test time, we sample a random z from the standard normal and pass that along with the observed state to get a sequence of actions. We can execute that sequence in its entirety or predict a sequence at each timestep and take a weighted average of the next action from the current predicted sequence and the k past predicted sequences.

Some notes about conditional VAEs:

1. We should pass in the entirety of what we want to predict (the full a) into the encoder q_ϕ at training time. Otherwise, we may not get good groupings. For instance, if a is a sequence and we pass in all but the last action and the last action is the right or left scenario, then we could face the same issue as the original problem with an unconditional VAE.
2. We can pass in more information than needed into the encoder at training time. For instance, we are trying to predict actions of an imitation learning dataset using the decoder, we can pass in both the state and actions into the encoder. We can try both and see which gives better results.
3. The prior can be conditioned on the input or not (it was not in Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware where we just used standard normal). It might make it easier for the model to shape the

conditional distribution produced by the encoder to make it more like the prior if we also have control over the prior, but it is not necessary.

4. We can pass in differing amounts of the input x to the encoder and decoder as long as what we pass to the encoder during training is available at test time. For instance, we could pass in just the state of a robot's joints to the encoder but pass in state of the joints plus what the robot sees to the decoder like in Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. We can also pass in more information such as all states in the sequence instead of just the first state like in Learning Latent Plans from Play. When passing less information to the encoder, we just have to make sure that the less information still allows groupings to be made effectively. For instance, if did not pass the observation of a tree into the decoder, the latent space would separate left and right actions less efficiently for that observation. e.g. they might still be separated but the chance they are separate when given a tree observation will be smaller since it will be harder for the model to learn. Note that when we pass in more information to the encoder, we can actually have the encoder predict that extra information (e.g. any extra input passed to the encoder can be thought of extra output we could technically predict). In Learning Latent Plans from Play, they could have made the model also predict states instead of just actions. We must also make sure that what we pass to the prior must be available at test time for sampling.

9 Learning Latent State Space from Observations

Besides using VAEs to model actions, we can also use them to learn how to generate the latent state from observations in a partially observed MDP, where we only have access to observations but not states. To do so, we will use a conditional sequence VAE that generates a sequence of states from a sequence of observations and actions. We can think of the input and output of the conditional VAE as the observations (e.g. we are trying to reconstruct the observations from the latent space of states), conditioned on the actions. Here, the output a is a sequence observations $o_{1:T}$ and the latent variable is a sequence of states $s_{1:T}$. We will also assume that $q_\phi(s_{1:T}|o_{1:T}, a_{1:T})$ can be decomposed as

$$q_\phi(s_{1:T}|o_{1:T}, a_{1:T}) = \prod_{t=1}^T q_\phi(s_t|o_{1:t}, a_{1:t-1})$$

Note that we can assume anything about the distribution q , but the more assumptions we make about q , the harder it is to minimize the gap between the ELBO and the log likelihood.

The ELBO used as the conditional sequence VAE loss is the loss where output a is a sequence observations $o_{1:T}$ and the latent variable is a sequence of states $s_{1:T}$

$$\begin{aligned} & \log(p(o_{1:T}|a_{1:T})) \\ & \geq ELBO \\ & = \mathbb{E}_{s_{1:T} \sim q_\phi(s_{1:T}|o_{1:T}, a_{1:T})} [\log p_\theta(s_{1:T}|o_{1:T}, a_{1:T}) + \log p_\theta(s_{1:T}|a_{1:T})] + \mathcal{H}(q_\phi(s_{1:T}|o_{1:T}, a_{1:T})) \\ & = \mathbb{E}_{s_{1:T} \sim q_\phi(s_{1:T}|o_{1:T}, a_{1:T})} [\log p_\theta(o_{1:T}|a_{1:T}, s_{1:T})] - D_{KL}(q_\phi(s_{1:T}|o_{1:T}, a_{1:T}) || p_\theta(s_{1:T}|a_{1:T})) \\ & = \mathbb{E}_{s_{1:T} \sim q_\phi(s_{1:T}|o_{1:T}, a_{1:T})} \left[\log \left(\prod_{t=1}^T p(o_t|a_{1:T}, s_{1:T}, o_{1:t-1}) \right) \right] - \mathbb{E}_{s_{1:T} \sim q_\phi(s_{1:T}|o_{1:T}, a_{1:T})} \left[\log \left(\frac{q_\phi(s_{1:T}|o_{1:T})}{p_\theta(s_{1:T}|a_{1:T})} \right) \right] \\ & = \mathbb{E}_{s_{1:T} \sim q_\phi(s_{1:T}|o_{1:T}, a_{1:T})} \left[\sum_{t=1}^T \log(p(o_t|s_t)) \right] - \mathbb{E}_{s_{1:T} \sim q_\phi(s_{1:T}|o_{1:T}, a_{1:T})} \left[\log \left(\prod_{t=1}^T \frac{q_\phi(s_t|o_{1:t}, a_{1:t-1})}{p_\theta(s_t|s_{t-1}, a_{t-1})} \right) \right] \\ & = \sum_{t=1}^T \mathbb{E}_{s_t \sim q_\phi(s_t|o_{1:t}, a_{1:t-1})} [\log(p(o_t|s_t))] - \mathbb{E}_{s_{1:T} \sim q_\phi(s_{1:T}|o_{1:T}, a_{1:T})} \left[\sum_{t=1}^T \log \left(\frac{q_\phi(s_t|o_{1:t}, a_{1:t-1})}{p_\theta(s_t|s_{t-1}, a_{t-1})} \right) \right] \\ & = \sum_{t=1}^T \mathbb{E}_{s_t \sim q_\phi(s_t|o_{1:t}, a_{1:t-1})} [\log(p(o_t|s_t))] - \sum_{t=1}^T \mathbb{E}_{s_{1:T} \sim q_\phi(s_{1:T}|o_{1:T}, a_{1:T})} \left[\log \left(\frac{q_\phi(s_t|o_{1:t}, a_{1:t-1})}{p_\theta(s_t|s_{t-1}, a_{t-1})} \right) \right] \\ & = \sum_{t=1}^T \mathbb{E}_{s_t \sim q_\phi(s_t|o_{1:t}, a_{1:t-1})} [\log(p(o_t|s_t))] - \sum_{t=1}^T \mathbb{E}_{s_t \sim q_\phi(s_t|o_{1:t}, a_{1:t-1}), s_{t-1} \sim q_\phi(s_{t-1}|o_{1:t-1}, a_{1:t-2})} \left[\log \left(\frac{q_\phi(s_t|o_{1:t}, a_{1:t-1})}{p_\theta(s_t|s_{t-1}, a_{t-1})} \right) \right] \\ & = \sum_{t=1}^T \mathbb{E}_{s_t \sim q_\phi(s_t|o_{1:t}, a_{1:t-1})} [\log(p(o_t|s_t))] - \sum_{t=1}^T \mathbb{E}_{s_{t-1} \sim q_\phi(s_{t-1}|o_{1:t-1}, a_{1:t-2})} \left[\mathbb{E}_{s_t \sim q_\phi(s_t|o_{1:t})} \left[\log \left(\frac{q_\phi(s_t|o_{1:t}, a_{1:t-1})}{p_\theta(s_t|s_{t-1}, a_{t-1})} \right) \middle| s_{1:t-1} \right] \right] \\ & = \sum_{t=1}^T \mathbb{E}_{s_t \sim q_\phi(s_t|o_{1:t}, a_{1:t-1})} [\log(p(o_t|s_t))] - \sum_{t=1}^T \mathbb{E}_{s_{t-1} \sim q_\phi(s_{t-1}|o_{1:t-1}, a_{1:t-2})} [D_{KL}(q_\phi(s_t|o_{1:t}, a_{1:t-1}) || p_\theta(s_t|s_{t-1}, a_{t-1}))] \\ & = \sum_{t=1}^T \mathbb{E}_{s_t \sim q_\phi(s_t|o_{1:t}, a_{1:t-1})} [\log(p_\theta(o_t|s_t))] - \mathbb{E}_{s_{t-1} \sim q_\phi(s_{t-1}|o_{1:t-1}, a_{1:t-2})} [D_{KL}(q_\phi(s_t|o_{1:t}, a_{1:t-1}) || p_\theta(s_t|s_{t-1}, a_{t-1}))] \end{aligned}$$

We can represent $q_\phi(s_t|o_{1:t}, a_{1:t-1})$ with an autoregressive model and $p_\theta(o_t|s_t)$ as a normal DNN. Note that if we use an RNN, we model

$$q_\phi(s_t|o_{1:t}, a_{1:t-1}) = q_\phi(s_t|o_t, s_{t-1}, a_{t-1})$$

If we use a transformer, we can either use both the encoder and decoder where we pass in $(o_{1:T}, a_{1:T})$ to the encoder or we can use just the decoder and pass in $(o_{1:T}, a_{1:T})$ as the preset input. Both set ups will have the decoder produce $s_{1:T}$ one time step at a time and would produce the following formulation

$$q_\phi(s_t|o_{1:t}, a_{1:t-1}) = q_\phi(s_t|o_{1:T}, a_{1:T}, s_{t-1})$$

Distributions are typically modeled as Gaussians with diagonal covariance matrices.

During test time, we discard the decoder (unlike most VAE use cases where we perform sampling) and use the encoder q_ϕ to get the latent state to use for planning. We also use the learned prior $p_\theta(s_t|s_{t-1}, a_{t-1})$ as our dynamics

model and replan at every time step. Notice how the prior is only trained through the KL loss rather than a loss used to predict the next latent state.

To further generalize, the **Learning Latent Dynamics for Planning from Pixels** paper also derives a bound where the prior is only allowed to look back starting from d (rather than 1) time steps before. This d step look back restriction objective is a lower bound on the objective given the prior is only allowed to look back starting from d (rather than 1) time steps before.

Besides using the learned latent states for model based RL with planning, we can also use it for things like policy gradients with actor critic.

Note instead of a variational autoencoder, we could probably also use a normal encoder without any sort of prior to get a latent state representation since we do not need to perform sampling. However, one benefit of the VAE is that we also get to train a dynamics model in parallel to help with model based RL.

10 Bayesian Neural Networks

Typically, the latent variable Z is used to model the data distribution X by producing a network that outputs $p_\theta(x|z)$ and an estimator $q_\phi(z|x)$ used to approximate $p(z|x)$. We can then sample from X by first sampling from Z and then sampling from $p_\theta(x|z)$ and can estimate the probability of a sample x by estimating

$$p(x) = \mathbb{E}_{z \sim p(z)} [p_\theta(x|z)] = \mathbb{E}_{z \sim q_\phi(z|x)} \left[\left(\frac{p(z)}{q_\phi(z|x)} \right) p_\theta(x|z) \right]$$

using Monte Carlo estimates (e.g. $\frac{1}{k} \sum_{i=1}^k p_\theta(x|z)$ if sampling from $p(z)$ or $\frac{1}{k} \sum_{i=1}^k \frac{p(z)}{q_\phi(z|x)} p_\theta(x|z)$ if sampling from $z \sim q_\phi(z|x)$).

Bayesian neural networks do something similar, but directly model $p_\theta(x)$ and use the model parameters as latent variables. However, instead of using this model for sampling or finding probabilities, it let's us estimate $q(\theta|\phi)$ where ϕ the parameters that define the distribution over θ learned for a particular dataset. Typically, we use a factorized Gaussian as the distribution over model parameters for $q(\theta|\phi)$, where ϕ is a vector of concatenated distribution parameters. In the factorized Gaussian case, each individual parameter θ_i (e.g. neural network weight) is parameterized as $\mathcal{N}(\mu_i, \sigma_i^2)$, and if there are n parameters ϕ would be a vector containing $[\mu_1, \sigma_1, \mu_2, \sigma_2, \dots, \mu_n, \sigma_n]$. In RL, these type models allow us to quantify information gain, which is defined as the KL divergence between the posterior and the prior

$$D_{KL}(p(z|y)||p(z))$$

This is equal to $D_{KL}(p(\theta|D)||p(\theta|D, (s_t, a_t, s_{t+1})))$ in RL, which is estimated by

$$D_{KL}(q(\theta|\phi')||q(\theta|\phi))$$

where ϕ is our best approximation the posterior distribution parameters of the model weights given the current data and ϕ' is our best approximation the posterior distribution parameters of the model weights given the current data and the new transition (s_t, a_t, s_{t+1}) appended to it. Note that in latent variable models, we have one amortized model $q(z|x)$ that produces distribution parameters of z for any x . Assuming the Gaussian case, the model would output (e.g. μ and ϕ). Instead of having a model, we could have also created a table containing the model parameters μ and ϕ for every possible x , which is similar to what we are doing here, where we have two inputs, the original data and the data with the new transition, and we just create separate sets of distribution parameters ϕ' and ϕ for each.

To estimate ϕ , we want to solve

$$\underset{\phi}{\operatorname{argmin}} D_{KL}(q(\theta|\phi)||p(\theta|D))$$

Note that $p_\theta(D) = ELBO + D_{KL}(q(\theta|\phi)||p(\theta|D))$, so minimizing the ELBO with respect to ϕ is the same as maximizing $D_{KL}(q(\theta|\phi)||p(\theta|D))$

$$\phi = \underset{\phi}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi)} [\log p(D|\theta)] - D_{KL}[q(\theta|\phi)||p(\theta)]$$

Similarly

$$\phi' = \underset{\phi}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log p(D, (s_t, a_t, s_{t+1})|\theta)] - D_{KL}[q(\theta|\phi')||p(\theta)]$$

The first term in the expression can be minimized using a Monte Carlo estimate and the reparameterization trick since $q(\theta|\phi)$ and $q(\theta|\phi')$ are Gaussians. The second term has a closed form if $p(\theta)$ is also Gaussian or it can also be minimized in the same fashion as the first expression. The prior $p(\theta)$ is typically also modeled as a Gaussian or mixture of Gaussians.

Another way to speed up finding ϕ' given ϕ is to notice that

$$\begin{aligned}
& \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log p(D, (s_t, a_t, s_{t+1})|\theta)] - D_{KL}[q(\theta|\phi')||p(\theta)] \\
&= \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log p(D, (s_t, a_t, s_{t+1})|\theta)] - \mathbb{E}_{\theta \sim q(\theta|\phi')} \left[\log \left(\frac{q(\theta|\phi')}{p(\theta)} \right) \right] \\
&= \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log p(D, (s_t, a_t, s_{t+1})|\theta)] - \mathbb{E}_{\theta \sim q(\theta|\phi')} \left[\log \left(\frac{q(\theta|\phi')}{\frac{p(\theta|D)p(D)}{p(D|\theta)}} \right) \right] \\
&= \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p((s_t, a_t, s_{t+1})|\theta)p(D|\theta))] - \mathbb{E}_{\theta \sim q(\theta|\phi')} \left[\log \left(\frac{q(\theta|\phi')}{p(\theta|D)p(D)} \right) \right] - \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p(D)|\theta)] \\
&= \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p((s_t, a_t, s_{t+1})|\theta))] + \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p(D|\theta))] - \mathbb{E}_{\theta \sim q(\theta|\phi')} \left[\log \left(\frac{q(\theta|\phi')}{p(\theta|D)p(D)} \right) \right] - \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p(D)|\theta)] \\
&= \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p((s_t, a_t, s_{t+1})|\theta))] - \mathbb{E}_{\theta \sim q(\theta|\phi')} \left[\log \left(\frac{q(\theta|\phi')}{p(\theta|D)p(D)} \right) \right] \\
&= \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p((s_t, a_t, s_{t+1})|\theta))] - \mathbb{E}_{\theta \sim q(\theta|\phi')} \left[\log \left(\frac{q(\theta|\phi')}{p(\theta|D)} \right) \right] + \mathbb{E}_{\theta \sim q(\theta|\phi')} [p(D)] \\
&= \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p((s_t, a_t, s_{t+1})|\theta))] - \mathbb{E}_{\theta \sim q(\theta|\phi')} \left[\log \left(\frac{q(\theta|\phi')}{p(\theta|D)} \right) \right] + p(D) \\
&= \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p((s_t, a_t, s_{t+1})|\theta))] - \mathbb{E}_{\theta \sim q(\theta|\phi')} \left[\log \left(\frac{q(\theta|\phi')}{p(\theta|D)} \right) \right] \\
&= \underset{\phi'}{\operatorname{argmin}} \mathbb{E}_{\theta \sim q(\theta|\phi')} [\log (p((s_t, a_t, s_{t+1})|\theta))] - D_{KL}[q(\theta|\phi')||p(\theta|D)]
\end{aligned}$$

Since $p(\theta|D)$ is intractable, we instead can replace it with $q(\theta|\phi)$ (this will be faster, although a bit less accurate). The VIME paper uses this approximated objective within a trajectory to get the $q(\theta|\phi')$ for each encountered (s_t, a_t, s_{t+1}) (note that we do not update $q(\theta|\phi)$ within each trajectory.) $q(\theta|\phi)$ is only updated at the end of each trajectory.

Algorithm 1: Variational Information Maximizing Exploration (VIME)

for each epoch n **do**

for each timestep t in each trajectory generated during n **do**

 Generate action $a_t \sim \pi_\alpha(s_t)$ and sample state $s_{t+1} \sim \mathcal{P}(\cdot|\xi_t, a_t)$, get $r(s_t, a_t)$.

 Add triplet (s_t, a_t, s_{t+1}) to FIFO replay pool $\mathcal{R}$.

 Compute $D_{\text{KL}}[q(\theta; \phi'_{n+1}) \| q(\theta; \phi_{n+1})]$ by approximation $\nabla^\top H^{-1} \nabla$, following Eq. (16) for diagonal BNNs, or by optimizing Eq. (12) to obtain ϕ'_{n+1} for general BNNs.

 Divide $D_{\text{KL}}[q(\theta; \phi'_{n+1}) \| q(\theta; \phi_{n+1})]$ by median of previous KL divergences.

 Construct $r'(s_t, a_t, s_{t+1}) \leftarrow r(s_t, a_t) + \eta D_{\text{KL}}[q(\theta; \phi'_{n+1}) \| q(\theta; \phi_{n+1})]$, following Eq. (7).

 Minimize $D_{\text{KL}}[q(\theta; \phi_n) \| p(\theta)] - \mathbb{E}_{\theta \sim q(\cdot; \phi_n)} [\log p(\mathcal{D}|\theta)]$ following Eq. (6), with $\mathcal{D}$ sampled randomly from $\mathcal{R}$, leading to updated posterior $q(\theta; \phi_{n+1})$.

 Use rewards $\{r'(s_t, a_t, s_{t+1})\}$ to update policy π_α using any standard RL method.

Figure 5: VIME Paper Algorithm

Still need to do PPO, GAE, and backprop through time, Info gain region of support, vertical and horizontal lines (copy picture)

10.1 Model Uncertainty

These types of models can also be used to decrease epistemic uncertainty (e.g. model uncertainty), that is caused by the variance (in the method) term in the bias variance decomposition of the error of a random point. This type of error decreases as we collect more data. The other type of uncertainty is aleatoric uncertainty, where the data itself may be stochastic due to randomness in the environment or noise when collecting data. This type of error does not go down as we collect more data and is the irreducible error in the bias-variance decomposition.